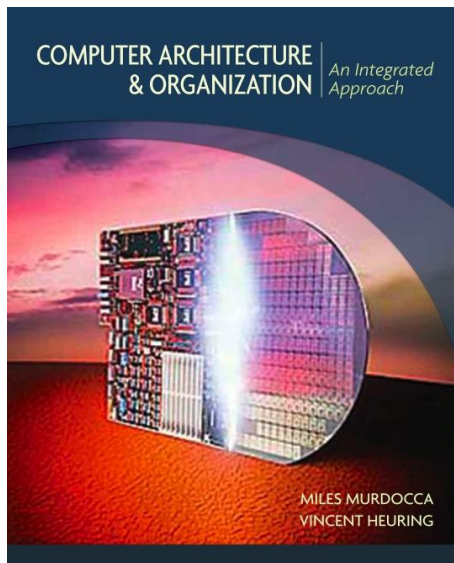


Organización de las Computadoras

Carolina Chaves Garcia



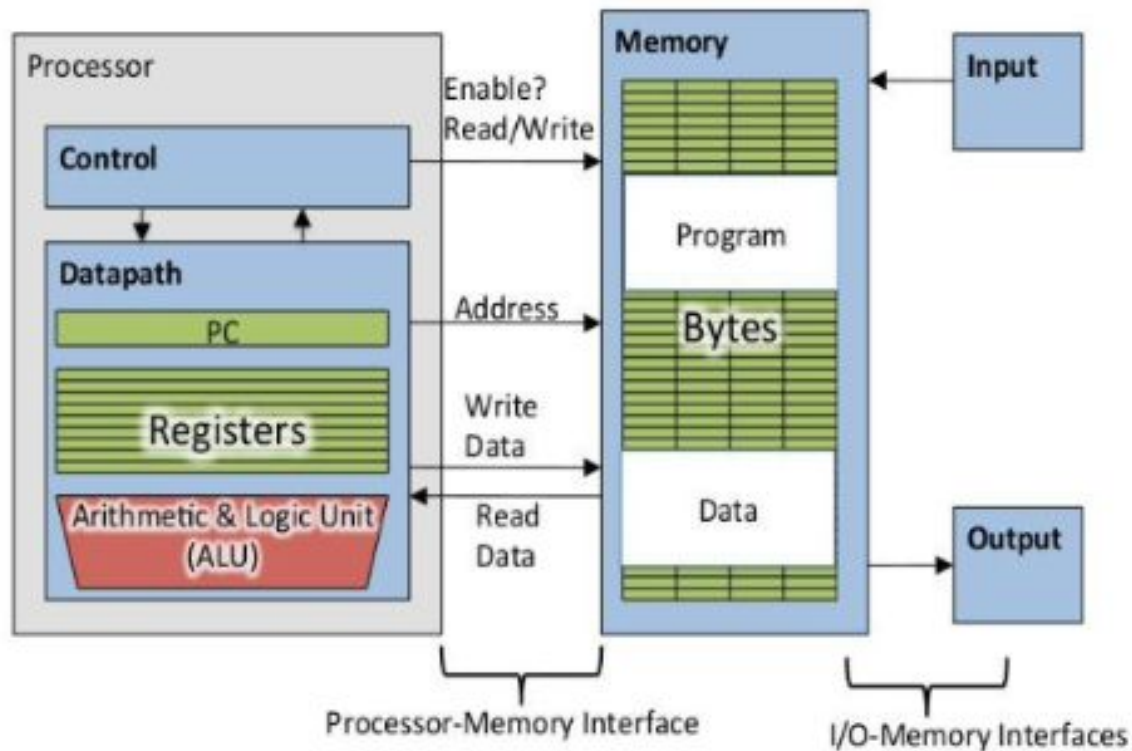
Programación Directa

Introducción

Introducción

Computadora

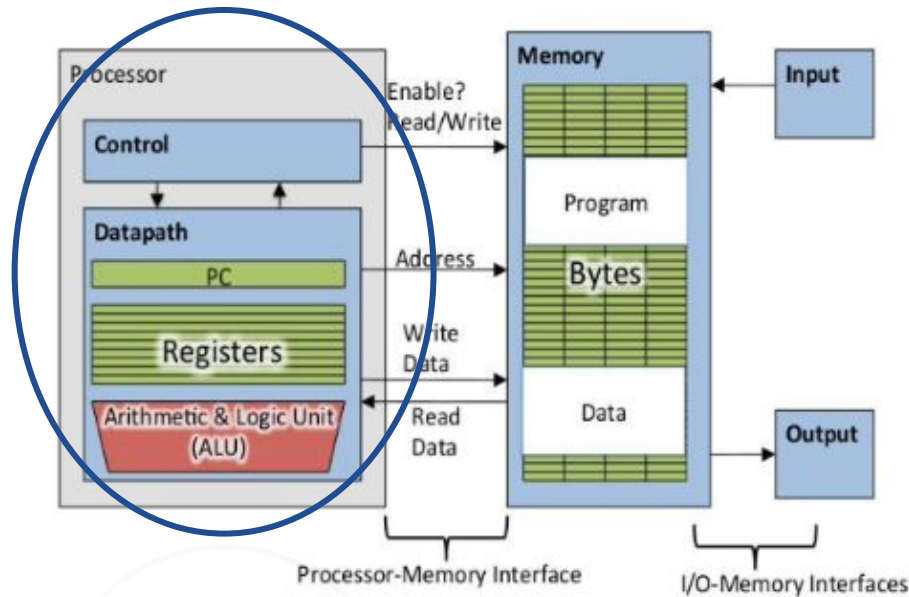
- **Máquina electrónica que procesa datos.**
- Podemos decir que es una máquina secuencial síncrona.
- **Lee, interpreta y ejecuta programas.**
- **Programa = instrucciones almacenadas.**



Introducción

Unidad Central de Procesamiento (CPU)

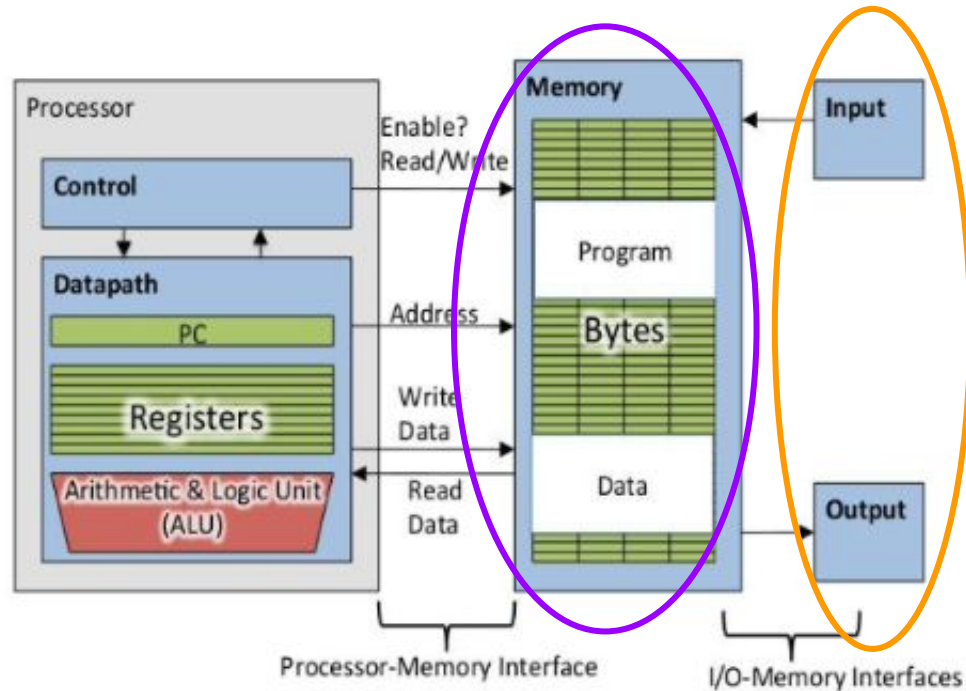
- Ejecuta las instrucciones y realiza los cálculos.
- Se compone de
 - Unidad de Control (UC): Coordina las operaciones, decodifica instrucciones y maneja el flujo de datos.
 - Unidad Aritmético-Lógica (ALU): Realiza operaciones matemáticas (suma, resta, etc.) y lógicas (AND, OR, comparaciones).
 - Registros: Pequeñas memorias ultrarrápidas que almacenan datos temporales (ej: operandos, direcciones de memoria)



Introducción

Unidad Central de Procesamiento (CPU)

- Memoria (RAM, ROM, caché) Memoria Almacenamiento de datos e instrucciones
- Dispositivos de Entrada/Salida (E/S): Comunicación con el mundo
 - Entrada: Teclado, mouse, micrófono.
 - Salida: Monitor, impresora, altavoces.
- Buses (comunicación entre componentes): Autopista de datos (direcciones, datos, control)

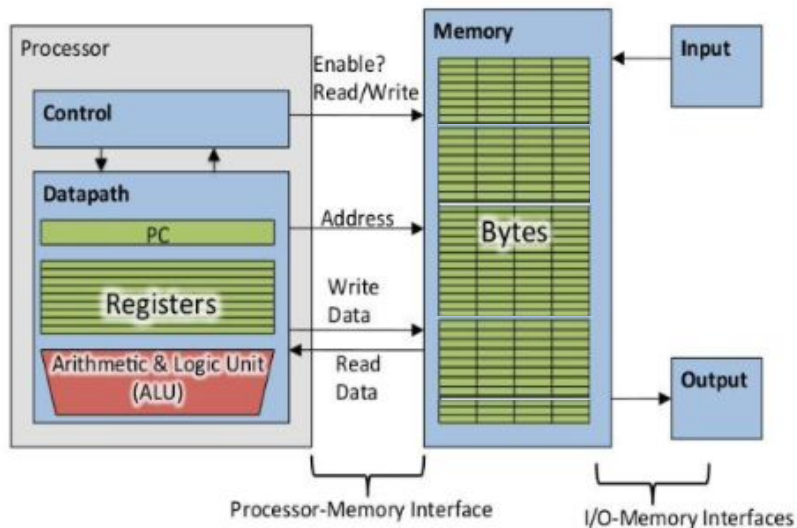


Introducción

Arquitecturas principales

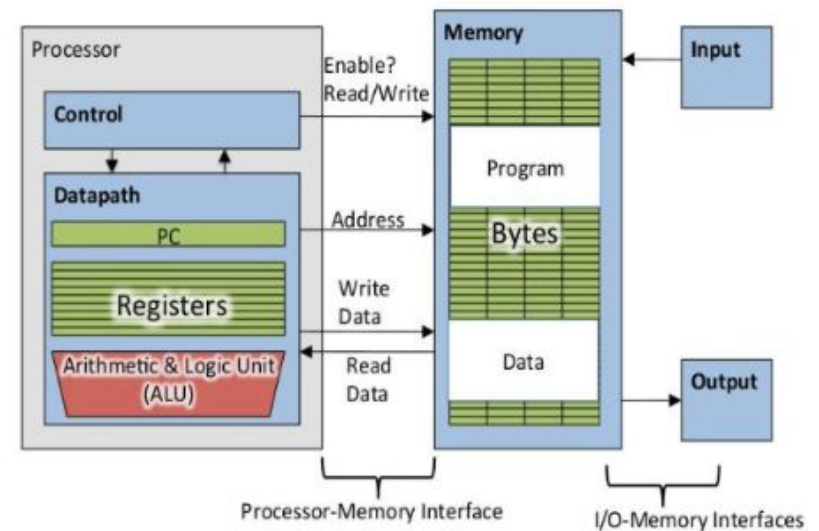
- Von Neumann

- Misma memoria para datos e instrucciones
- PCs modernos



- Harvard

- Memorias separadas para datos e instrucciones
- Microcontroladores (PIC, ESP32-C3)



Instrucciones

Instrucciones

ISA = Instruction Set Architecture

- Conjunto de reglas y comandos que define como un programa interactúa con el hardware de la CPU.
- RV32I - Tenemos 47 instrucciones
- Cada instrucción es una palabra de 32 bits almacenada en little-endian
- Tipos de instrucciones
 - Tipo R: Dos registros fuente y uno destino
 - Tipo I: Un registro fuente y una constante (inmediato)
 - Tipo S: Almacena datos desde registro a memoria
 - Tipo B: Saltos condicionales. Similar al formato S, pero con un inmediato ajustado para direccionamiento.
 - Tipo U: Cargar constantes grandes en registros (20 bits superiores).
 - Tipo J: Saltos incondicionales largos
- Operaciones comunes: R; I; S
- Manejo de direccionamiento: B;J
- Control de flujo: U

Instrucciones

ISA = Instruction Set Architecture

- Campos:

- OpCode: Código que identifica el tipo de instrucción = 7 bits
- Registros:
- rs1 (Read Source 1): Registro fuente.
- rs2 (Read Source 2): Registro fuente 2.
- rd (Register Destination): Registro de destino.
- Funciones: Subclasifica la operación.
- Inmediatos: Almacena una constante o direcciones de memoria.

Formato	Bit																															
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R (Register/register)	funct7							rs2					rs1					funct3			rd				opcode							
Nº de bits	7							5					5					3			5				7							
I (Immediate)	imm[11:0]													rs1					funct3			rd				opcode						
Nº de bits	12													5					3			5				7						
S (Store)	imm[11:5]							rs2					rs1					funct3			imm[4:0]				opcode							
Nº de bits	7							5					5					3			5				7							
B (Branch)	[12]	imm[10:5]						rs2					rs1					funct3			imm[4:1]			[11]	opcode							
Nº de bits	1	6						5					5					3			4			1	7							
U (Upper immediate)	imm[31:12]																				rd				opcode							
Nº de bits	20																				5				7							
J (Jump)	[20]	imm[10:1]										[11]	imm[19:12]									rd				opcode						
Nº de bits	1	10										1	8									5				7						

Instrucciones

Instrucciones de cálculo - Entre registros

Tipo R	Operación	Mnemonic	Descripción
Aritméticas	Suma	add rd, rs1, rs2	$rd \leftarrow rs1 + rs2$
	Resta	sub rd, rs1, rs2	$rd \leftarrow rs1 - rs2$
	AND	and rd, rs1, rs2	$rd \leftarrow rs1 \wedge rs2$
	OR	or rd, rs1, rs2	$rd \leftarrow rs1 \vee rs2$
	XOR	xor rd, rs1, rs2	$rd \leftarrow rs1 \vee rs2$
Comparaciones	Set if less than	slt rd, rs1, rs2	$rd \leftarrow rs1 < rs2 \text{ 1:0}$
	Set if less than unsigned	sltu rd, rs1, rs2	$rd \leftarrow rs1 < rs2 \text{ 1:0}$
Desplazamientos	Shift left logic	sll rd, rs1, rs2	$rd \leftarrow rs1 \ll rs2$
	Shift right logic	srl rd, rs1, rs2	$rd \leftarrow rs1 \gg rs2$
	Shift right arithmetic	sra rd, rs1, rs2	$rd \leftarrow rs1 \gg rs2$

Instrucciones

Instrucciones de cálculo - Con constantes

Tipo I	Operación	Mnemonic	Descripción
Aritméticas	Suma	addi rd, rs1, lnm	$rd \leftarrow rs1 + lnm$
	AND	andi rd, rs1, lnm	$rd \leftarrow rs1 \wedge lnm$
	OR	ori rd, rs1, lnm	$d \leftarrow rs1 \vee lnm$
	XOR	xori rd, rs1, lnm	$rd \leftarrow rs1 \vee lnm$
Comparaciones	Set if less than	slti rd, rs1, lnm	$rd \leftarrow rs1 < lnm \text{ 1:0}$
Desplazamientos	Shift left logic	slli rd, rs1, lnm	$rd \leftarrow rs1 \ll lnm$
	Shift right logic	srli rd, rs1, lnm	$rd \leftarrow rs1 \gg lnm$
	Shift right arithmetic	srai rd, rs1, lnm	$rd \leftarrow rs1 \gg lnm$

Instrucciones

Instrucciones de lectura y almacenamiento

	Operación	Instrucción	Descripción
Tipo I			
Lectura	Leer byte	lb rd, offset(rs1)	rd <- [rs1 + signExt(offset)]
	Leer media palabra	lh rd, offset(rs1)	rd <- [rs1 + signExt(offset)]
	Leer palabra	lw rd, offset(rs1)	rd <- [rs1 + signExt(offset)]
	Leer byte sin signo	lbu rd, offset(rs1)	rd <- [rs1 + signExt(offset)]
	Leer media palabra sin signo	lhu rd, offset(rs1)	rd <- [rs1 + signExt(offset)]
Tipo S			
Almacenamiento	Guardar byte	sb rs2, offset(rs1)	Mem[rs1+offset] <- rs2(7,0)
	Guardar media palabra	sh rs2, offset(rs1)	Mem[rs1+offset] <- rs2(16,0)
	Guardar palabra	sw rs2, offset(rs1)	Mem[rs1+offset] <- rs2

Instrucciones

Instrucciones de transferencia de control

Tipo SB	Operación	Instrucción	Condición
Control de flujo	Saltar si igual	beq rs1, rs2, offset	Si $rs1 == rs2$, $pc += sext(inm)$
	Saltar si no igual	bne rs1, rs2, offset	Si $rs1 \neq rs2$, $pc += sext(inm)$
	Saltar si menor	blt rs1, rs2, offset	Si $rs1 < rs2$, $pc += sext(inm)$
	Saltar si mayor o igual	bge rs1, rs2, offset	Si $rs1 \geq rs2$, $pc += sext(inm)$
	Saltar si menor sin signo	bltu rs1, rs2, offset	Si $rs1 < rs2$ (unsigned), $pc += sext(inm)$
	Saltar si mayor o igual sin signo	bgeu rs1, rs2, offset	Si $rs1 \geq rs2$ (unsigned), $pc += sext(inm)$
Salto y enlace	Saltar al registro y enlazar	jalr rd, offset(rs1)	$rd \leftarrow pc + 4$, $pc \leftarrow rs1 + sext(inm, 1'b0)$
	Guardar palabra	sw rs2, offset(rs1)	$Mem[rs1+offset] \leftarrow rs2$

Instrucciones

Otras instrucciones especiales

	Operación	Instrucción	Descripción
Tipo U	Cargar inmediato alto	lui rd, offset	$rd \leftarrow \{inm20, 12'b0\}$
	Sumar inmediato alto a PC	auipc rd, offset	$rd \leftarrow pc + \{inm20, 12'b0\}$
Tipo UJ	Saltar y enlazar	jal rd, offset	$rd \leftarrow pc + 4, pc \leftarrow pc + sext(inm20)$

Modos de direccionamiento

Modos de direccionamiento

Los modos de direccionamiento definen cómo una instrucción accede a los operandos en memoria o registros. Son fundamentales para optimizar código y mejorar la eficiencia del programa.

Modo inmediato:

- Operando dentro de la instrucción.
- El operando está directamente en la instrucción.

`addi x5, x0, 10` `# x5 = 10` (Carga inmediata de 10 en el registro x5)

Modo de registro:

- Los operandos están en registros.

`add x6, x5, x4` `# x6 = x5 + x4`

Modo indirecto:

- Uso de registros para apuntar a direcciones de memoria.
- El registro contiene la dirección de memoria donde se encuentra el dato.

`lw x12, 0(x11)` `# x12` obtiene el valor almacenado en la dirección contenida en `x11`

Modos de direccionamiento

Modo directo (absoluto):

- Dirección de memoria especificada.
- La instrucción contiene la dirección de memoria del dato.
- No es común en RISC-V, pero se puede simular con la (load address).

`la x10, valor # Cargar la dirección de 'valor' en x10`

`lw x11, 0(x10) # Cargar el dato de la memoria en x11`

- `la` (Load Address - Cargar Dirección)
 - La instrucción `la` se usa para cargar la dirección de memoria de una variable en un registro.
- `lw` (Load Word - Cargar una Palabra de Memoria)
 - La instrucción `lw` se usa para leer un valor almacenado en memoria y guardarlo en un registro.

Modos de direccionamiento

Modo indexado:

- Uso de desplazamientos para acceder a posiciones de memoria.
- Útil para manejar arreglos y estructuras de datos.

```
addii x13,x0, 4      # Tamaño de un entero
mul x14, x7, x13     # Calcula el desplazamiento (índice * 4)
lw x15, 0(x14)       # Carga el elemento del array
```

Modo relativo al PC

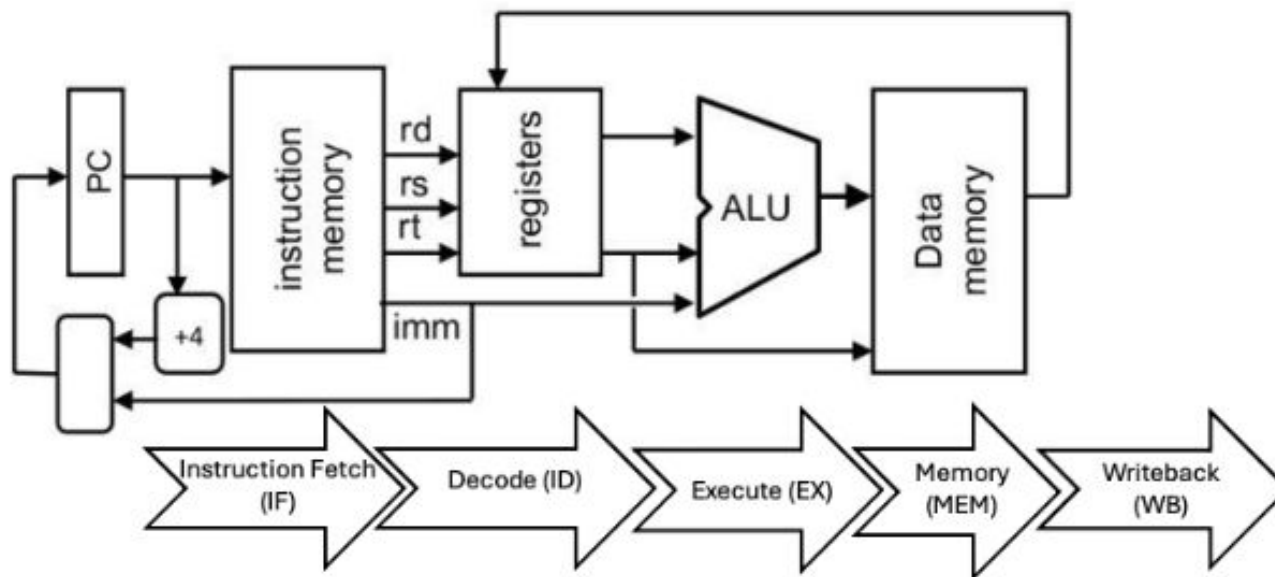
- Utilizado para saltos y llamadas a funciones.

```
beq x5, x0, etiqueta    # Si x5 es 0, salta a 'etiqueta'
```

Datapath

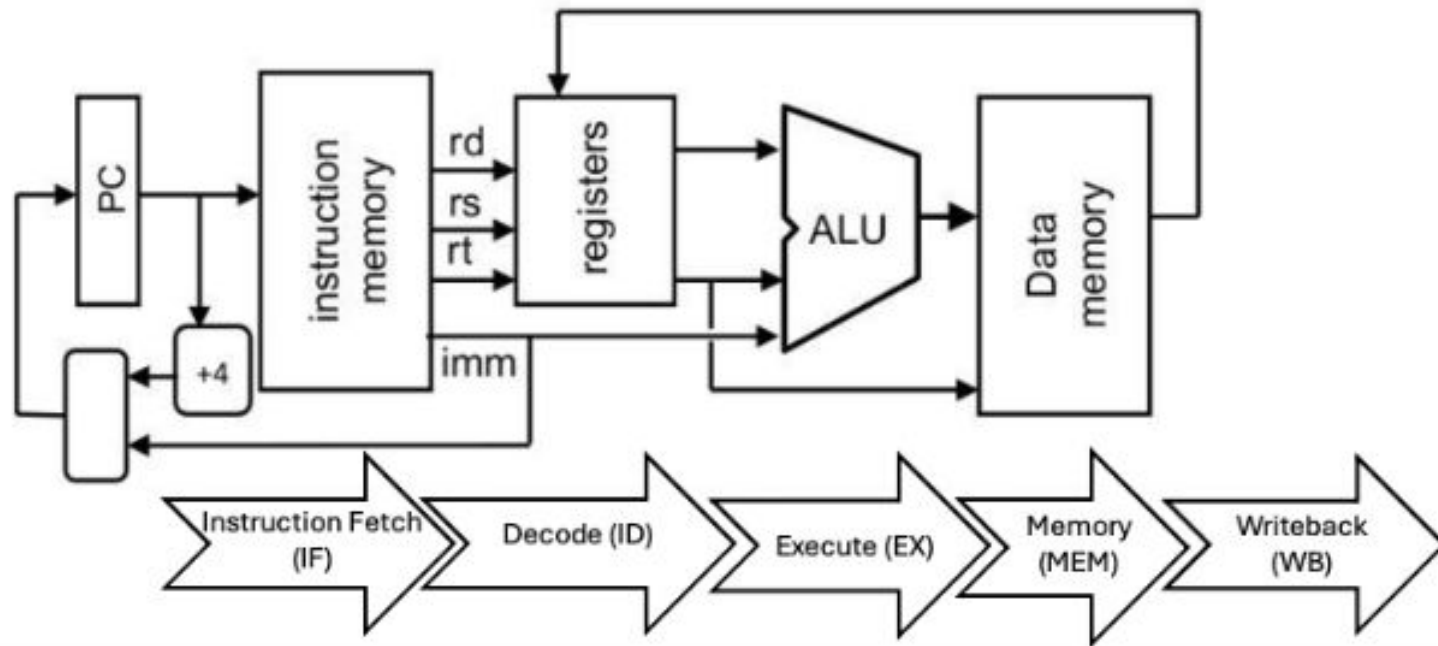
Datapath

- Es el conjunto de componentes hardware que:
 - **Transportan datos** (entre registros, ALU, memoria, etc.).
 - **Ejecutan operaciones** (cálculos, accesos a memoria).
 - **Siguen el ciclo Fetch-Decode-Execute**.
- Para acelerar el proceso, las CPUs modernas dividen el datapath en **etapas de pipeline** (como una cadena de montaje):
- **Pipeline:** La UC maneja varias instrucciones simultáneamente (ej: mientras una se decodifica, otra se ejecuta).



Datapath

1. **IF (Fetch):**
 1. **PC** envía la dirección de la instrucción a Instruction Memory.
 2. La CPU trae la instrucción desde la memoria
2. **ID (Decode): Interpreta la instrucción**
3. **EX (Execute):** La ALU realiza la operación.
4. **MEM (Acceso a memoria)**
5. **WB (Writeback):** Guarda el resultado en memoria o registros.



Ejemplos

Ejemplos

1. Cargar un número a un registro.
2. Sumarle una constante a ese número que cargamos.
3. Guardar el resultado en otro registro.
4. Suma de dos números: Cargar dos números , sumarlos, y guardar el resultado en otra posición de memoria.

Gracias